

WORLD KARATE · BACKEND

World Karate Backend

Complete architecture, API, data, security,
and deployment guide

Architecture, API, data model, operations, and deployment guide Document edition: 1.0 · Code reviewed: 23 June 2026 · Application version: 1.0.0

World Karate Backend is the server-side application for the World Karate course platform. It exposes a JSON API for account registration and login, password recovery, profile maintenance, public course discovery, administrator course management, purchased-course access, newsletter registration, and Zarinpal checkout and payment verification.

This guide documents what the current repository actually implements. Where the schema or dependencies suggest unfinished functionality, the difference is called out under Known limitations.

Contents

- At a glance
- Quick start
- Configuration
- Architecture
- Data model
- Authentication and authorization
- Functional behavior
- API reference
- Payment lifecycle
- Validation and error behavior
- Security model
- Build, deployment, and operations
- Testing and troubleshooting
- Known limitations

At a glance

Area	Implementation
Runtime	Node.js, Express 4, TypeScript compiled to CommonJS
Persistence	PostgreSQL through Prisma 5
Authentication	JWT stored in an HTTP-only auth-token cookie
Passwords	bcrypt, configurable cost factor
Validation	Zod request schemas
Payments	Zarinpal SDK; sandbox in development
HTTP hardening	Helmet, CORS, body-size limit, cookie parser
Main entities	User, Course, Transaction, Newsletter, two join entities
API style	Route-oriented JSON endpoints; Persian user-facing messages
Tests	No automated test suite is currently configured

Implemented capabilities

- Register a user with a unique email and bcrypt-hashed password.
- Log in and receive a cookie-based JWT; log out by overwriting that cookie.
- Start password recovery, validate a four-digit OTP, and reset the password.
- Update the authenticated user's name and email, then refresh the JWT.
- List all courses or retrieve one course publicly.
- Let administrators create/delete courses and query a user's courses.
- List courses owned by the authenticated user.
- Register unique newsletter email addresses.
- Build a course basket from server-side prices, create an unpaid Zarinpal transaction, verify payment, and grant course ownership idempotently.

Quick start

Prerequisites

- Node.js 20 or newer is recommended. The repository was successfully built during this review with Node 22.17.1 and npm 11.16.0.
- PostgreSQL reachable through a Prisma connection URL.
- Zarinpal merchant credentials. They are required at application startup, even if payment endpoints are not called.

1. Install dependencies

```
npm install
```

The postinstall script runs prisma generate automatically.

2. Configure the environment

Create .env in the repository root. Never commit real secrets.

```
DATABASE_URL="postgresql://USER:PASSWORD@HOST:5432/DATABASE?schema=public"
JWT_SECRET="replace-with-a-long-random-secret"
PORT=3000
ROUNDS=10
ZARINPAL_MERCHANT_ID="your-merchant-id"
ZARINPAL_ACCESS_TOKEN="your-access-token"
FRONTEND_URL="http://localhost:5173"
BACKEND_URL="http://localhost:3000"
```

ENV_MODE is set by the npm scripts, so it normally does not need to be stored in .env.

3. Create or update the database

For development, apply migrations and regenerate the client:

```
npx prisma migrate dev
npx prisma generate
```

For a deployed database, apply committed migrations without creating a new one:

```
npx prisma migrate deploy
```

4. Run in development

```
npm run dev
```

Nodemon runs app.ts with ENV_MODE=development. The default API address is http://localhost:3000.

5. Build and run production output

```
npm run build
npm start
```

The build emits JavaScript and source maps to dist/; npm start executes dist/app.js with ENV_MODE=production.

6. Create the first administrator

There is no admin-creation endpoint. Register a normal user, then promote it directly in the database using a controlled administrative process:

```
UPDATE "User" SET "isAdmin" = true WHERE email = 'admin@example.com';
```

Configuration

Variable	Required	Purpose	Default/current behavior
DATABASE_URL	Yes	PostgreSQL connection used by Prisma	No application default
JWT_SECRET	Yes	Signs and verifies authentication JWTs	No safe default
ZARINPAL_MERCHANT_ID	Yes	Identifies the Zarinpal merchant	Startup throws if absent
ZARINPAL_ACCESS_TOKEN	Yes	Authenticates Zarinpal SDK requests	Startup throws if absent
PORT	No	Express listening port	3000
ROUNDS	No	bcrypt work factor for registration/reset	10
FRONTEND_URL	No	Base URL used to create /payment/verify callback	http://localhost:3000
BACKEND_URL	No	Present in the local environment convention	Not referenced by current code
ENV_MODE	Via scripts	development selects sandbox and relaxed cookie/CORS behavior	dev/start scripts set it

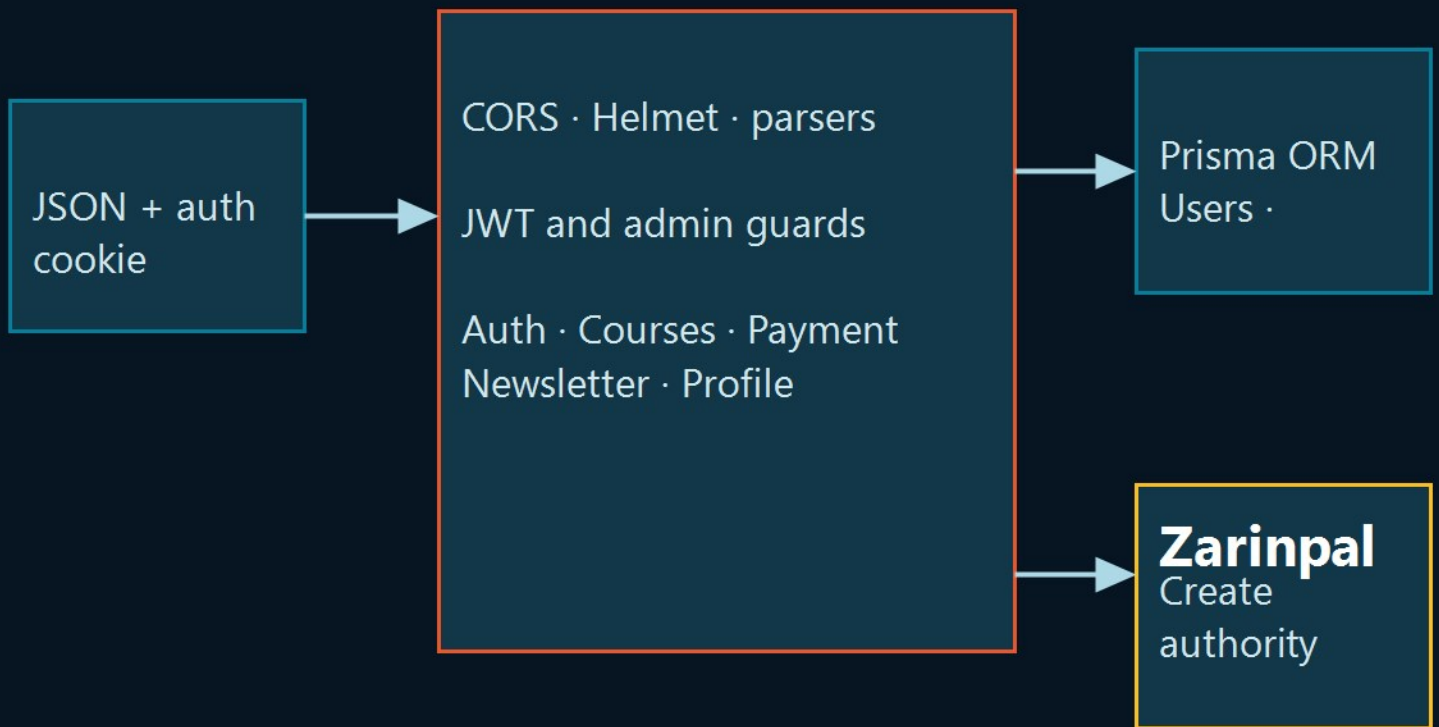
Production mode has application-specific consequences:

- CORS accepts only <https://worldkarate.ir> and <https://www.worldkarate.ir>, with credentials enabled.
- The auth cookie becomes Secure, remains SameSite=Lax, and receives domain worldkarate.ir.
- Zarinpal uses production mode and payment URLs under <https://www.zarinpal.com/pg/StartPay>.

Development mode accepts reflected origins (origin: true), uses a non-secure host cookie, and selects Zarinpal sandbox URLs.

Architecture

Runtime Architecture



Database state is authoritative for identity, roles, prices, and course

The application is a single-process modular monolith. `app.ts` configures cross-cutting middleware and mounts small Express routers. Feature routers call a shared Prisma client directly; there is no separate service or repository layer. This keeps the code compact, but transaction boundaries and reusable business rules must currently be handled in each route.

Source layout

<code>app.ts</code>	Application composition and route mounting
<code>middleware/</code>	
<code>authorization.ts</code>	JWT cookie verification and user hydration
<code>adminAuth.ts</code>	Database-backed administrator guard
<code>prisma/</code>	
<code>schema.prisma</code>	Canonical data model
<code>migrations/</code>	PostgreSQL migration history
<code>db.ts</code>	Shared <code>PrismaClient</code> instance
<code>schemas/</code>	Zod request validation
<code>src/</code>	
<code>auth/</code>	Registration, login, recovery, profile, logout
<code>courses/</code>	Public, user, and administrator course routes
<code>newsletter/</code>	Subscriber registration
<code>payment/</code>	Zarinpal checkout and verification
<code>utils/</code>	JWT, cookies, environment, OTP and SDK helpers
<code>dist/</code>	Compiled production JavaScript

Request pipeline

Every request passes through CORS, JSON/urlencoded parsing, Helmet, and cookie parsing. Protected endpoints then run authorization:

- Read auth-token from cookies.
- Verify its signature with `JWT_SECRET`.
- Find the current user by the token's email.
- Attach the full database user to `req.body.user`.
- Continue to the route; administrator routes additionally query `isAdmin` from the database.

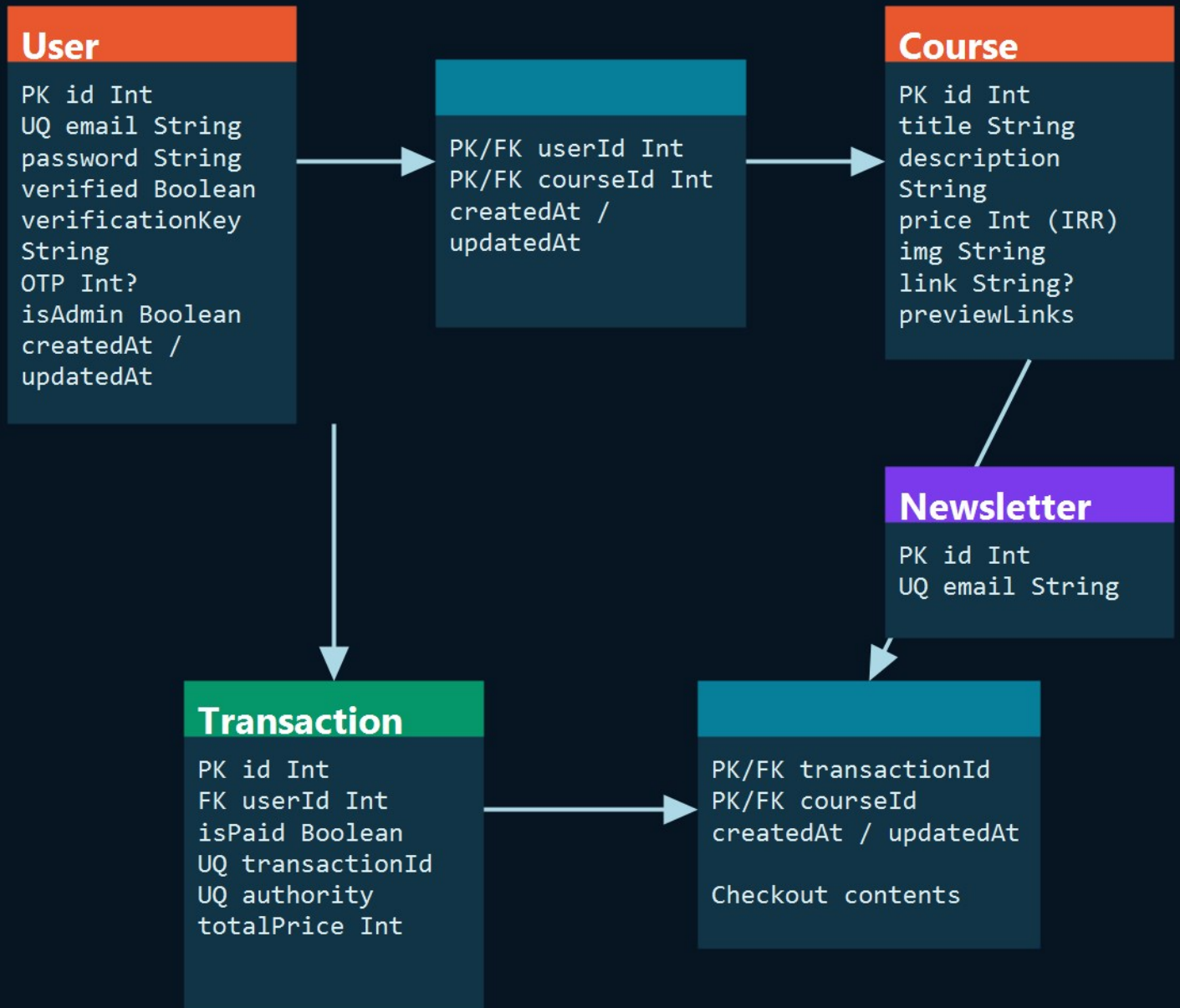
Reloading the user makes database state authoritative. The client cannot grant itself admin rights merely by changing a request body or stale token.

External boundaries

- PostgreSQL stores all durable application state.
- Zarinpal creates payment authorities and verifies completed payments.
- The web frontend owns navigation to the hosted gateway and submits the authority to the verification endpoint after returning to its `/payment/verify` page.
- Email delivery is not currently integrated. Although Nodemailer is installed, no mailer is called.

Data model

Entity Relationship Diagram



Entity responsibilities

Entity	Responsibility and important constraints
User	Identity, password hash, admin flag, recovery OTP, verification metadata. Email and verification key are unique.
Course	Public catalog record. Price is stored as an integer in Iranian rials. link is nullable in the database; previewLinks is a PostgreSQL text array.
Transaction	One checkout attempt. Belongs to one user, stores total, payment authority/reference identifiers, and paid state.
UsersOnCourses	Composite-key ownership relation. A user can own a course once. Created only after successful verification.
TransactionsOnCourses	Composite-key snapshot of which courses were included in a checkout attempt.
Newsletter	Independent unique email subscriber. It does not reference User.

Relationship semantics

- A user has zero or many transactions.
- Users and courses are many-to-many through UsersOnCourses.
- Transactions and courses are many-to-many through TransactionsOnCourses.
- Foreign keys use Prisma/PostgreSQL's restrictive deletion behavior. Deleting a course referenced by ownership or transaction rows can therefore fail until dependent records are handled.
- The transaction's join rows preserve basket membership. Its totalPrice preserves the charged total even if course prices later change.

Identifier nuance

At checkout, both authority and transactionId initially contain the Zarinpal authority. After successful verification, transactionId is replaced with Zarinpal's ref_id; authority remains the stable lookup key. API responses also use transactionId for the database row ID during checkout, so consumers should distinguish:

- checkout response transactionId: internal numeric transaction primary key;
- stored Transaction.transactionId: external authority before payment, external ref_id after payment.

Authentication and authorization

Cookie and token

The cookie is named `auth-token`, is HTTP-only, applies to `/`, and lasts 10 days. JavaScript in the browser cannot read it. Frontend requests must include credentials—for example, `fetch(..., { credentials: "include" })`.

The JWT contains `email`, `firstName`, `lastName`, and `isAdmin`. It has no explicit expiry; practical browser persistence is currently controlled by cookie lifetime. Profile updates create a new JWT so its identity claims remain current.

Registration and login

Registration validates the name, email, and password; checks email uniqueness; normalizes each name to lower case with its first character capitalized; hashes the password; and creates a random 83-character verification key. Registration does not log the user in.

Login intentionally returns the same generic message for an unknown email and a bad password. On success it sets the cookie and returns public profile fields.

Password recovery

The current recovery protocol is:

- `PUT /forget-password` validates the email, creates a four-digit OTP, stores it on the user, and returns it in JSON.
- `POST /validate-otp` compares the submitted numeric OTP with the stored value and, on success, sets the same authentication cookie used for normal login.
- `PUT /reset-password` is protected by that cookie, validates matching passwords, hashes the new password, and updates the authenticated user.

The OTP is not emailed, expired, single-use, rate-limited, or cleared after use in the current implementation. Treat this flow as development-stage until the mitigations in Known limitations are applied.

Authorization matrix

Capability	Public	Authenticated user	Administrator
Register/login/recovery/newsletter	✓	✓	✓
List/view courses	✓	✓	✓
Checkout and verify payment	verify is public; checkout requires auth	✓	✓
Update own profile/logout/list owned courses		✓	✓
Create/delete courses			✓
Query courses by user email			✓

Functional behavior

Course catalog and access

Public callers can list and inspect courses. A course contains marketing data, a price in rials, an image value, an optional full course link in the database, and preview link values. The authenticated ownership endpoint returns full course records whose UsersOnCourses relation contains the current user.

The create endpoint validates title, description, numeric nonnegative price, image, and link. Although Course.link is nullable in Prisma, the API schema currently requires it. The create route does not accept previewLinks.

Administration

Admin checks do not trust the JWT's isAdmin claim; they read the current database row. Admin endpoints create courses, delete a course by numeric ID, and attempt to list courses owned by a specified email.

Newsletter

Newsletter signup validates an email, rejects duplicates, and creates a standalone subscriber row. There are no list, unsubscribe, or campaign endpoints.

Payment and course granting

Checkout accepts course IDs, loads matching courses from the database, rejects an empty/invalid basket and already-owned items, then calculates the total from stored prices. The client cannot submit a price. A successful Zarinpal request creates an unpaid transaction and its course rows before returning a hosted payment URL.

Verification finds the transaction by authority and uses its stored total for server-to-server verification. Code 100 marks it paid, stores the reference ID, and inserts course ownership rows with duplicate skipping. An already-paid transaction returns success without repeating the grant operation.

API reference

All bodies and responses are JSON. Successful and error messages are predominantly Persian. Unless noted, unexpected failures return 500.

Endpoint index

Method	Path	Access	Purpose
POST	/register	Public	Create an account
POST	/login	Public	Authenticate and set cookie
POST	/logout	User	Clear auth cookie
PUT	/profile	User	Update own profile
PUT	/forget-password	Public	Generate recovery OTP
POST	/validate-otp	Public	Validate OTP and set cookie
PUT	/reset-password	User	Set new password
GET	/fetch-course	Public	List courses
GET	/fetch-course/:courseId	Public	Get one course
POST	/create-course	Admin	Create course
DELETE	/delete-course/:courseId	Admin	Delete course
GET	/admin/fetch-course/:email	Admin	Query a user's courses
GET	/user/fetch-course	User	List current user's courses
POST	/register-newsletter	Public	Subscribe an email
POST	/payment/checkout	User	Create checkout transaction
POST	/payment/verify	Public	Verify by authority and grant courses

Identity endpoints

POST /register

```
{
  "firstName": "Sara",
  "lastName": "Ahmadi",
  "email": "sara@example.com",
  "password": "karate123"
}
```

- Names: 3–20 letters/spaces.
- Password: at least 8 characters with at least one lowercase Latin letter and one digit.
- Returns 200 with verificationKey; duplicate email returns 409; invalid data returns 400.

POST /login

```
{ "email": "sara@example.com", "password": "karate123" }
```

Returns 200, sets auth-token, and includes user.firstName, user.lastName, and user.email. Invalid input returns 400; invalid credentials return 403.

POST /logout

Requires the auth cookie. Overwrites it with an empty value using the standard cookie options and returns 200.

PUT /profile

Any of the fields may be supplied; email is optional despite its schema's wording.

```
{ "firstName": "سارا", "lastName": "احمدی", "email": "new@example.com" }
```

Names allow Persian or Latin letters without spaces and are 3–20 characters. Returns 200 and refreshes the JWT. A conflicting email currently falls through to 500.

PUT /forget-password

```
{ "email": "sara@example.com" }
```

Returns 200 with { message, OTP }. Invalid and unknown emails intentionally receive the same Persian message, with status 400 and 404 respectively.

POST /validate-otp

```
{ "email": "sara@example.com", "OTP": 4831 }
```

OTP must be a JSON number from 1000 through 9999. Success sets auth-token; invalid input or mismatch returns 400.

PUT /reset-password

```
{ "newPassword": "newpass123", "repeatPassword": "newpass123" }
```

Requires the auth cookie. Both values must satisfy the password policy and match. Returns 200.

Course endpoints

GET /fetch-course

Returns 200 with an array of all course rows. There is no pagination, sorting, filtering, or projection.

GET /fetch-course/:courseId

Returns one course or 404. A malformed numeric ID reaches the Prisma call and is handled as 400.

POST /create-course

```
{
  "title": "Advanced Kumite",
  "description": "A complete advanced kumite training program.",
  "price": 2500000,
  "img": "https://cdn.example.com/kumite.jpg",
  "link": "https://courses.example.com/kumite"
}
```

Requires user and admin guards. Title is 5–50 characters; description is 20–1000; price is a nonnegative JSON number. img and link are required strings but are not URL-validated. Returns 200 with the created course.

DELETE /delete-course/:courseId

Requires admin access. Returns 200; unknown/malformed IDs and foreign-key restrictions are currently returned as 500.

GET /user/fetch-course

Returns all course records owned by the authenticated user.

GET /admin/fetch-course/:email

Requires admin access. Returns 400 for an unknown user. Current defect: the implementation maps `userId` instead of `courseId` from the join rows, so the returned courses can be incorrect.

Newsletter endpoint

POST /register-newsletter

```
{ "email": "reader@example.com" }
```

Returns 200; invalid or already-registered emails return 400.

Payment endpoints

POST /payment/checkout

```
{ "courseIds": ["1", "2"] }
```

The route expects an array and converts each value with `Number`. It has no Zod schema, so `omit/null/wrong-type` values may become server errors. Valid course matches—not requested count—define the basket; nonexistent IDs are silently excluded if at least one valid course remains. Duplicate requested IDs are collapsed by the database query.

Success response:

```
{
  "message": "در حال انتقال به درگاه پرداخت...",
  "paymentUrl": "https://sandbox.zarinpal.com/pg/StartPay/A000...",
  "authority": "A000...",
  "transactionId": 42
}
```

Empty/wholly invalid/already-owned baskets or a rejected gateway request return 400.

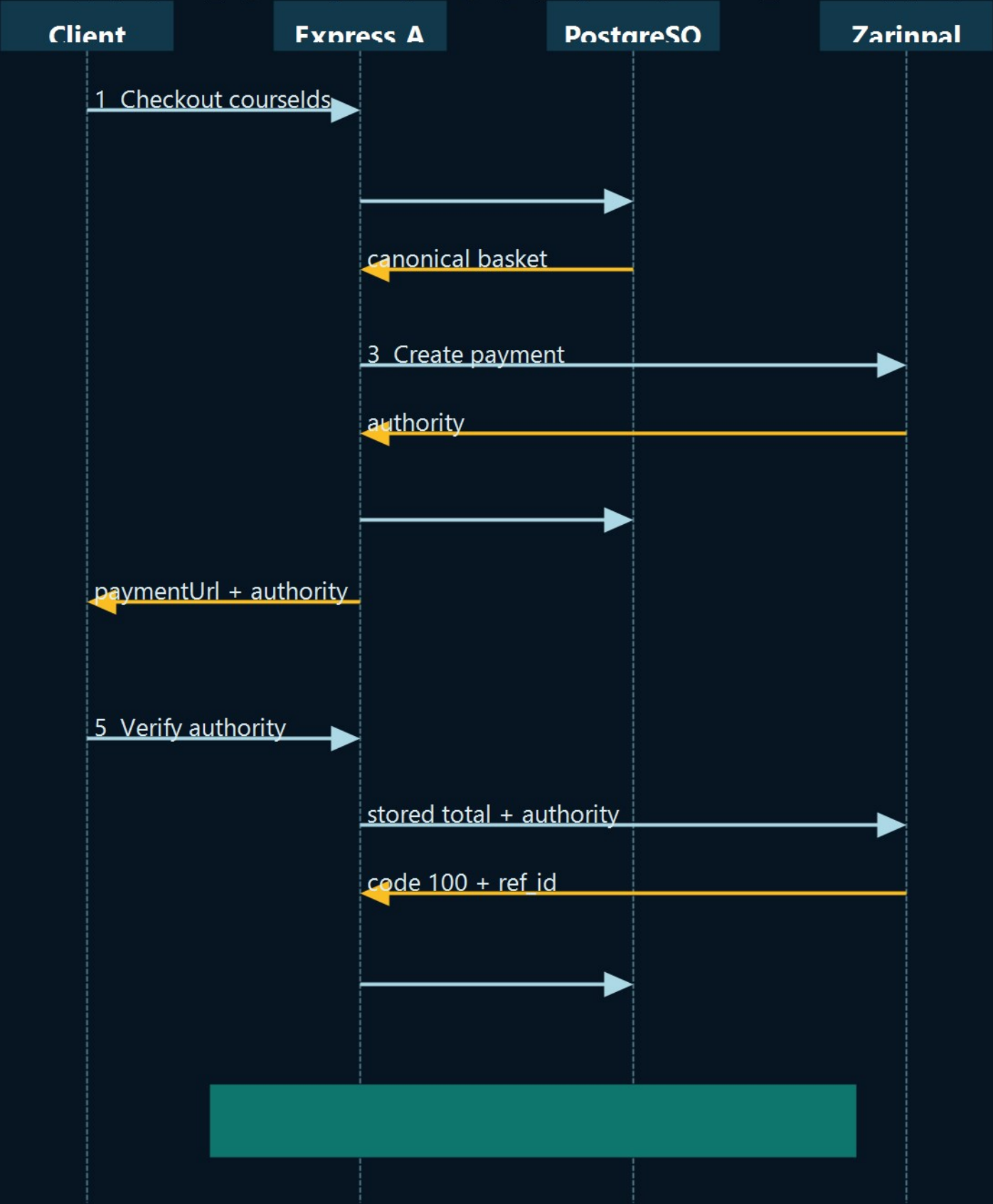
POST /payment/verify

```
{ "authority": "A000..." }
```

Returns 404 when no transaction matches. On gateway code 100, returns 200, marks the transaction paid, and grants courses. Already-paid transactions also return 200. Other gateway results return 400. The route is public because it acts as a payment completion endpoint; possession of a valid authority identifies the transaction.

Payment lifecycle

Course Purchase and Verification



There are two durable relations for a reason:

- TransactionsOnCourses answers “what was this checkout attempting to buy?”
- UsersOnCourses answers “what may this user access now?”

Only successful verification copies the transaction's course IDs into ownership. This separation prevents an unpaid checkout from granting access. The already-paid branch and the ownership composite key make repeated verification largely idempotent.

The transaction update and ownership grant are currently separate database operations, not a single Prisma transaction. A process/database failure between them can produce a paid transaction without all course grants; operations should have a reconciliation path until they are made atomic.

Validation and error behavior

Zod validates registration, login, password recovery/reset, profile updates, and course creation. Payment inputs and numeric URL parameters are validated manually or implicitly.

Common status conventions:

Status	Meaning in this API
200	Successful create/update/read; the API does not currently use 201
400	Validation, invalid basket/payment, OTP mismatch, or malformed course lookup
403	Bad login or access denied (subject to middleware chaining issue below)
404	Unknown course, recovery email, or transaction
409	Registration email already exists
500	Unhandled Prisma, gateway, or server failure

Prisma error objects are sometimes included directly in JSON. Production APIs should normalize errors and avoid exposing internal details.

Security model

Existing controls

- bcrypt password hashing; hashes are never returned by normal route responses.
- HTTP-only authentication cookie and production-only Secure flag.
- JWT signature verification plus a fresh database user lookup on protected requests.
- Database-backed administrator check.
- Helmet security headers.
- Credentialed production CORS allowlist.
- Server-side payment total calculation and server-to-server payment verification.
- Unique/composite database keys prevent duplicate accounts, newsletter entries, authorities, and course ownership.
- Generic login failures reduce account enumeration through that endpoint.

Operational security requirements

- Generate a high-entropy `JWT_SECRET`; rotate it through a planned forced-login process.
- Store `.env` outside version control and use the deployment platform's secret manager.
- Serve production only over HTTPS; the cookie will not work over plain HTTP in production mode.
- Restrict database network access, use TLS where appropriate, and back up PostgreSQL.
- Protect Zarinpal credentials and separate sandbox from production credentials.
- Add request-rate limits to login, registration, OTP generation/validation, newsletter, and payment endpoints.
- Do not log full gateway payloads or sensitive error objects in production.

Build, deployment, and operations

npm commands

Command	Result
npm run dev	Starts Nodemon against TypeScript with development mode
npm run build	Runs tsc and writes to dist/
npm start	Runs compiled app in production mode
npm test	Placeholder that exits with failure; no tests exist
npx prisma generate	Regenerates Prisma client after schema/dependency changes
npx prisma migrate dev	Develops/applies migrations locally
npx prisma migrate deploy	Applies committed migrations in deployment
npx prisma studio	Opens a local database browser; never expose publicly

The project compiled successfully with npm run build during documentation generation.

Production checklist

- Provision PostgreSQL and set a least-privileged application user.
- Supply all required secrets and set the correct FRONTEND_URL.
- Run npm ci, which also generates Prisma Client.
- Run npx prisma migrate deploy as a controlled release step.
- Run npm run build and start node dist/app.js (or npm start).
- Place the process behind an HTTPS reverse proxy/load balancer.
- Confirm the frontend origin is one of the hard-coded production CORS origins and that cookie domain routing is valid.
- Add process supervision, structured logs, health/readiness endpoints, metrics, alerts, and database backups.
- Perform a Zarinpal production checkout with a low-value test course and verify ownership creation.

Scaling characteristics

The API is stateless apart from PostgreSQL and the external gateway, so multiple process replicas can share the same database. Cookie JWTs do not require sticky sessions. Before scaling writes, make payment finalization atomic and consider connection limits or a database pooler; every Node process owns a Prisma connection pool.

Testing and troubleshooting

Suggested smoke test

- Register, log in, and confirm the auth-token cookie is stored.
- Promote the test user to admin and create a course.
- Fetch the public course list and single course.
- Start checkout in sandbox, complete the gateway flow, and submit the authority to verification.
- Confirm Transaction.isPaid=true and that GET /user/fetch-course contains the course.
- Call verification again and confirm it returns the already-paid success response without duplicate ownership.

Common failures

Symptom	Likely cause / action
App throws before listening	Missing Zarinpal merchant ID/access token; they are checked at module import
Prisma cannot connect	Check DATABASE_URL, database reachability, credentials, and applied migrations
Browser is not authenticated	Use credentials: "include"; verify HTTPS, CORS origin, cookie domain, and ENV_MODE
Development callback points to wrong app	Set FRONTEND_URL; its fallback is http://localhost:3000
Payment opens production/sandbox unexpectedly	Check ENV_MODE; exact value development enables sandbox
Course deletion returns 500	The course may be referenced by ownership/transaction join rows
Admin user-course response looks wrong	Known userId/courseId mapping defect
Prisma types differ from schema	Run npx prisma generate, then rebuild

Known limitations and recommended next steps

These are prioritized by risk and impact, not cosmetic preference.

Critical security and correctness

- Complete password recovery. Send the OTP through an actual mail provider; never return it in JSON. Store a hash, expiry, attempt count, and purpose; clear/rotate it after use. A recovery token should authorize password reset only, not become a general login cookie.
- Add JWT expiry and token lifecycle controls. Supply `expiresIn`, define rotation/revocation behavior, and invalidate existing sessions after password changes where required.
- Fix middleware response order. `res.json(...).status(403)` sends the default status before setting 403; use `res.status(403).json(...)`. The missing-user branch has the same problem. Type `findUnique` results safely before destructuring.
- Make payment finalization atomic. Wrap transaction state update and ownership grants in `prisma.$transaction`; define behavior for gateway success followed by a local failure.
- Validate payment input strictly. Add Zod for a nonempty, unique array of positive integer IDs and reject the entire basket if any requested ID is missing.
- Fix admin user-course lookup. Map `courseId`, not `userId`, or query `Course` with a relational filter as the user endpoint does.

API and operational maturity

- Decide whether account verification is a feature; implement and enforce `verified/verificationKey`, or remove them.
- Add CSRF protection or a deliberate same-site strategy for state-changing cookie-authenticated requests. `SameSite=Lax` helps but is not a complete policy for every deployment topology.
- Add rate limiting, consistent error envelopes, request IDs, structured logs, and health/readiness endpoints.
- Do not serialize raw exception/Prisma objects to clients. Map uniqueness, not-found, and foreign-key errors to safe 4xx responses.
- Validate `img/link` as URLs; support `previewLinks` if it is part of the product; reconcile the API-required link with the nullable database field.
- Add pagination and response projections. Public course responses currently include all stored fields, including full course links.
- Add an authenticated payment history/transaction status endpoint and an administrative reconciliation workflow.
- Add newsletter unsubscribe/consent metadata if messages will be sent.
- Add automated unit, integration, authorization, and payment idempotency tests; wire `npm test` to them and run them in CI.

Canonical sources: behavior was derived from `app.ts`, `src/`, `middleware/`, `schemas/`, `utils/`, `prisma/schema.prisma`, migration SQL, `package.json`, and `tsconfig.json`. If this guide and the executable code diverge, treat the code and applied database migrations as runtime truth, then update this document in the same change.